

# MedSupplyDrones

## Natural Language Command Centre

CW3 Explanation Document

*Student ID: s2141930*

Course: Informatics Large Practical (ILP)  
Academic Year: 2024/25

December 1, 2025

# 1 Innovation, Idea, and Benefit

## 1.1 Motivation and Customer Needs

As sole designer and developer of the MedSupplyDrones Natural Language Command Centre, I set out to address a practical barrier in medical logistics: non-technical staff often need to interact with highly technical REST endpoints to plan urgent drone deliveries. Dispatchers should not need to understand JSON schemas, coordinate systems, or capability constraints to perform their job, yet many workflows still demand this.

My goal was to make drone dispatching feel as natural as giving instructions to a colleague. A user should be able to write: *"Deliver 3L cooled insulin to the Royal Infirmary - minimise cost"* rather than assemble a complex schema. By lowering this barrier, the system provides immediate benefits: faster emergency responses, fewer errors, and drastically reduced cognitive load for the people who depend on medical drone logistics.

## 1.2 Novelty and Key Innovations

The novelty lies in how natural language, live ILP data, and structured planning logic are fused into a single, coherent tool. Several elements push this beyond a basic LLM wrapper:

- **Data-informed LLM reasoning:** The model receives up-to-date fleet capabilities, service points, restrictions. Every answer is grounded in real ILP data.
- **What-if strategy analysis:** Users can immediately compare strategies side-by-side (e.g., cheapest vs fastest), without multiple manual API calls.
- **Conversational fleet insights:** Users may ask questions like "Which drones can handle both cooled and heated deliveries?" and receive structured, data-driven answers.

These innovations make dispatching intuitive, reliable, and significantly quicker.

Quantitatively, planning time is estimated to reduce from 5-10 minutes (manual JSON construction with coordinate lookups) to approximately 30 seconds (natural language input), representing roughly 90% time savings. The project meaningfully improves both speed and quality of emergency decision-making.

## 2 Execution and Implementation

### 2.1 Full-Stack Overview (Frontend, Mid-Layer, Backend)

I designed each layer intentionally to complement the others:

- **Frontend (Streamlit UI)**

The frontend handles all UI. Streamlit enables a reactive Python-based interface without a separate JavaScript stack. I designed a clean layout where users can issue natural language instructions, view structured output, and perform what-if analyses.

- **Mid-Layer (LLM Orchestration + Parsing + Validation)**

This layer converts human language into structures dispatch requests using:

- prompt-engineered JSON schemas and examples,
- Pydantic validation,
- internal location-coordinate mapping,
- strategy conversion into CW2-compatible strategies.

This mid-layer ensures that the LLM remains grounded, predictable, and safe. Location parsing specifically works through: (1) LLM system prompts containing five pre-defined Edinburgh locations with coordinates (Appleton Tower, Royal Infirmary, Western General, St John's Hospital, Sick Kids Hospital), and (2) a fallback geocoding mechanism using a hardcoded lookup table plus Nominatim API for unknown addresses when the LLM response lacks coordinates.

- **Backend (FastAPI Service Layer)**

The backend exposes a clean API consumed by the Streamlit client, with endpoints for LLM parsing, plan construction, strategy comparison, and fleet queries. ILP and CW2 services sit behind dedicated client classes, keeping the system testable and maintainable.

The CW2 backend itself is a well-structured Spring Boot application (ilpCw1Application) with clear layers: REST controllers, a focused service tier, client/repository adapters, and an A\* pathfinding module. CW2 components enforce strong validation, and retrieve live ILP data via a configurable RestTemplate. I implemented an additional @postmapping: calcMultiDroneDeliveryPathAsGeoJson in the Java backend to allow for more complex delivery request combinations. The planner blends pragmatic heuristics (based on euclidean distance) with full A\* search, giving the Java side both performance and clean extensibility.

This tri-layer architecture spreads responsibility cleanly and avoids coupling LLM behaviour directly to UI components or external services.

### 2.2 Choice of Technologies

I selected Python as the end-to-end implementation language for its rapid development capabilities and its ecosystem support for:

- **LLMs:** through the `google-generativeai` SDK,
- **APIs:** via FastAPI's async capabilities,
- **Validation:** through Pydantic models mirroring CW2 schemas,
- **UI development:** through Streamlit's Python-native reactive components.

Gemini-2.5-flash was chosen for its reliable, structured output and latency performance (less than 2s). Prompt engineering includes pre-loading known Edinburgh location coordinates directly into system prompts, JSON schema definitions with examples, strategy mapping rules, and constraint handling instructions. This ensures the LLM consistently produces valid, schema-compliant JSON that passes Pydantic validation before reaching backend services.

Docker and Docker Compose give me reproducible builds and isolate environment differences, while strong typing helps catch issues early.

### **2.3 Engineering Practices and Patterns**

I adopted industry-standard practices:

- Client pattern for all ILP and CW2 interactions,
- Strategy pattern for optimisation options,
- Orchestrator layer to coordinate planning operations,
- Fully typed codebase using Pydantic and Python type hints,
- Error handling with user-facing clarity and developer-friendly logs,
- Automated tests for major endpoints and health checks.

This ensures the system remains maintainable, extendable, and testable.

### 3 Completeness

The implementation addresses all anticipated user workflows:

- **Natural language dispatching:** capacity, cooling/heating, cost constraints, multi-dispatch instructions.
- **Fleet Queries:** capabilities, availability, capacity distribution, service point assignments.
- **What-If strategy analysis:** cost, moves, and qualitative differences across strategies.
- **Manual JSON mode:** retained for expert users or debugging.

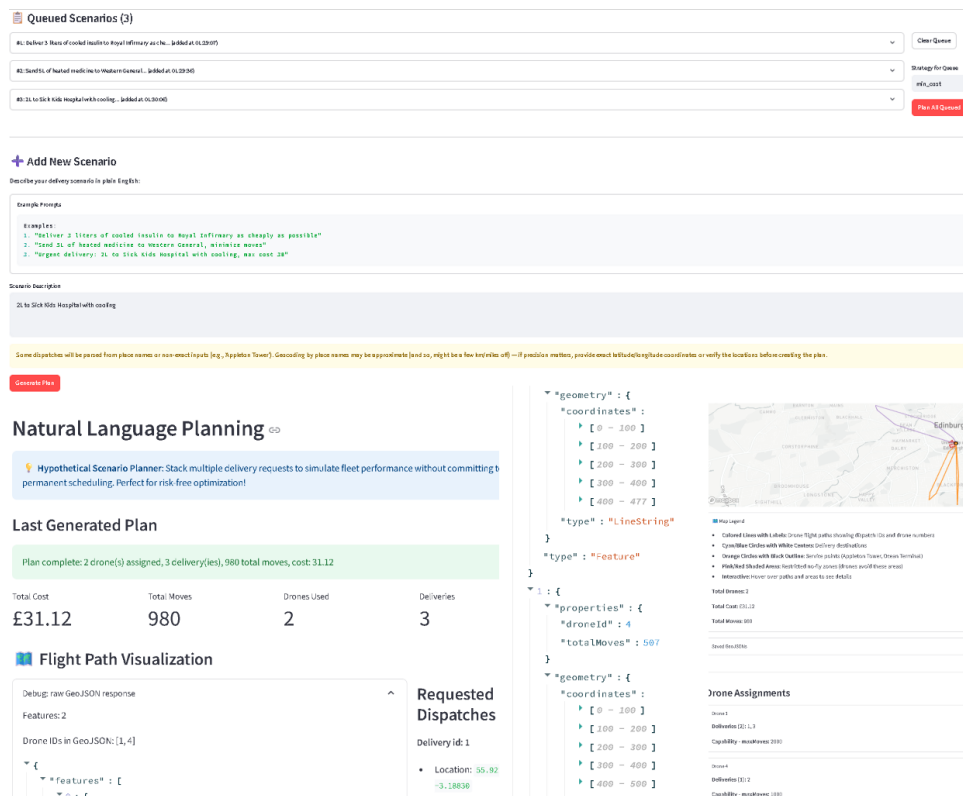


Figure 1: Natural language dispatching workflow for a multi-drone, multi-delivery dispatch request

I tested 40+ diverse natural language dispatch and fleet query phrasings, edge cases, unexpected inputs, and invalid location names. The system responds gracefully with clear guidance.

The project supports both simple one-off dispatches and complex multi-step plans, covering the full breadth of ILP requirements.

## What-If Strategy Comparison

Compare different planning strategies for the same dispatches.

Generate a plan first or manually enter JSON below

Dispatches JSON

```
[{"id":1,"requirements":{"capacity":3.0,"cooling":true},"delivery":{"lng":3.1883,"lat":55.9217}},
{"id":2,"requirements":{"capacity":3.0,"heating":true},"delivery":{"lng":3.2415,"lat":55.9542}},
{"id":3,"requirements":{"capacity":2.0,"cooling":true},"delivery":{"lng":3.2029,"lat":55.9233}}]
```

Strategy A

min\_cost

Strategy B

min\_moves

Compare Strategies

Comparison complete!

### Comparison Summary

Comparing min\_cost vs min\_moves:

- min\_moves costs 11.54 MORE
- min\_moves requires 31 FEWER moves

#### Plan A: min\_cost

Cost

£28.75

Moves

1011

Drones

2

#### Plan B: min\_moves

Cost

£40.29

↑ +11.54

Moves

980

↓ -31

Drones

2

### Fleet Capabilities Query

Refresh Fleet Data

Total Drones

10

With Cooling

4

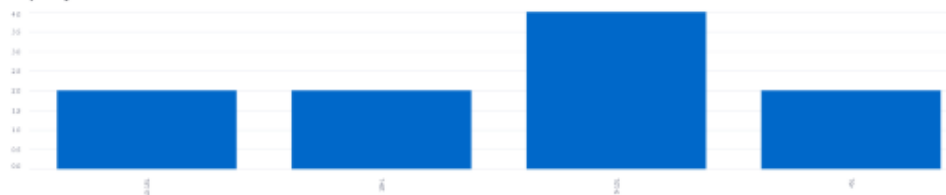
With Heating

7

Service Points

2

#### Capacity Distribution



#### Drones per Service Point

```
{
  "1" : 0,
  "2" : 0
}
```

### Ask a Question

Question

Which drones have both heating and cooling capability?

Ask

Delay Answering

3 drones with cooling & heating: Drone 1, Drone 9, Drone 8

Figure 2: What-if Strategy Comparison for minimising moves, minimising cost and balanced approaches & Fleet Capabilities Query with general drone data and question box for more detailed logistic inquiry

## 4 Style and Architecture

System structure using clean architecture principles:

- **Separation of concerns:** UI, LLM logic, planning logic, and external services are fully isolated.
- **SOLID design:** each module is responsible for a single task.
- **Abstraction over implementation:** the planner works over client interfaces, not hard-coded endpoints.
- **Typed, validated dataflow:** errors are caught before they propagate.
- **Maintainable code style:** PEP 8 adherence, clear naming, consistent formatting, and comprehensive docstrings.

Version control contains meaningful commits documenting each development phase. Environment variables avoid hardcoded secrets, and dependency pinning ensures reproducibility.

### 4.1 Unique Contributions

As sole developer, I concentrated on engineering quality despite the scope:

- prompt-engineering to enforce deterministic JSON output (system prompts include complete JSON schemas, example inputs/outputs, five Edinburgh location coordinates, strategy mapping rules, constraint handling instructions sent to Gemini),
- rich error-handling with fallback geocoding via Nominatim API when LLM responses lack coordinates,
- multi-deployment support including Dockerised orchestration,
- clean separation between LLM parsing (`llm_parser.py`) and plan execution (`planner.py`).

### 4.2 Future Enhancements

Potential extensions include:

- real-time map tracking,
- multi-turn conversational memory,
- predictive analytics based on weather or historical delivery performance,
- voice-based dispatching,
- database-backed analytics and history,
- mobile application integration,
- advanced multi-drone scheduling with time windows and battery constraints.